

Instruction-tuning

```
In [1]: ▶ import itertools
import jsonlines

from datasets import load_dataset
from pprint import pprint

from llama import BasicModelRunner
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
```

Load instruction tuned dataset

```
In [2]: ▶ instruction_tuned_dataset = load_dataset("tatsu-lab/alpaca", split="train",
```

Downloading readme: 0.00B [00:00, ?B/s]

```
In [3]: ► m = 5
print("Instruction-tuned dataset:")
top_m = list(itertools.islice(instruction_tuned_dataset, m))
for j in top_m:
    print(j)
```

Instruction-tuned dataset:

```
{ 'instruction': 'Give three tips for staying healthy.', 'input': '', 'output': '1.Eat a balanced diet and make sure to include plenty of fruits and vegetables. \n2. Exercise regularly to keep your body active and strong. \n3. Get enough sleep and maintain a consistent sleep schedule.', 'text': 'Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n#### Instruction:\nGive three tips for staying healthy.\n\n#### Response:\n1.Eat a balanced diet and make sure to include plenty of fruits and vegetables. \n2. Exercise regularly to keep your body active and strong. \n3. Get enough sleep and maintain a consistent sleep schedule.' }

{ 'instruction': 'What are the three primary colors?', 'input': '', 'output': 'The three primary colors are red, blue, and yellow.', 'text': 'Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n#### Instruction:\nWhat are the three primary colors?\n\n#### Response:\nThe three primary colors are red, blue, and yellow.' }

{ 'instruction': 'Describe the structure of an atom.', 'input': '', 'output': 'An atom is made up of a nucleus, which contains protons and neutrons, surrounded by electrons that travel in orbits around the nucleus. The protons and neutrons have a positive charge, while the electrons have a negative charge, resulting in an overall neutral atom. The number of each particle determines the atomic number and the type of atom.', 'text': 'Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n#### Instruction:\nDescribe the structure of an atom.\n\n#### Response:\nAn atom is made up of a nucleus, which contains protons and neutrons, surrounded by electrons that travel in orbits around the nucleus. The protons and neutrons have a positive charge, while the electrons have a negative charge, resulting in an overall neutral atom. The number of each particle determines the atomic number and the type of atom.' }

{ 'instruction': 'How can we reduce air pollution?', 'input': '', 'output': 'There are a number of ways to reduce air pollution, such as shifting to renewable energy sources, encouraging the use of public transportation, prohibiting the burning of fossil fuels, implementing policies to reduce emissions from industrial sources, and implementing vehicle emissions standards. Additionally, individuals can do their part to reduce air pollution by reducing car use, avoiding burning materials such as wood, and changing to energy efficient appliances.', 'text': 'Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n#### Instruction:\nHow can we reduce air pollution?\n\n#### Response:\nThere are a number of ways to reduce air pollution, such as shifting to renewable energy sources, encouraging the use of public transportation, prohibiting the burning of fossil fuels, implementing policies to reduce emissions from industrial sources, and implementing vehicle emissions standards. Additionally, individuals can do their part to reduce air pollution by reducing car use, avoiding burning materials such as wood, and changing to energy efficient appliances.' }

{ 'instruction': 'Describe a time when you had to make a difficult decision.', 'input': '', 'output': 'I had to make a difficult decision when I was working as a project manager at a construction company. I was in charge of a project that needed to be completed by a certain date in order to meet the client's expectations. However, due to unexpected delays, we were not able to meet the deadline and so I had to make a difficult decision. I decided to extend the deadline, but I had to stretch the team's resources even further and increase the budget. Although it was a risky decision, I ultimately decided to go ahead with it to ensure that the project was completed on time and that the client's expectations were met. The project was eventually successfully completed and this was seen as a testament to my leadership and decision-making abilities.', 'text': 'Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\n#### Instruction:\nDescribe a time when you had to make
```

a difficult decision.\n\n### Response:\nI had to make a difficult decision when I was working as a project manager at a construction company. I was in charge of a project that needed to be completed by a certain date in order to meet the client's expectations. However, due to unexpected delays, we were not able to meet the deadline and so I had to make a difficult decision. I decided to extend the deadline, but I had to stretch the team's resources even further and increase the budget. Although it was a risky decision, I ultimately decided to go ahead with it to ensure that the project was completed on time and that the client's expectations were met. The project was eventually successfully completed and this was seen as a testament to my leadership and decision-making abilities.'}

Two prompt templates

```
In [4]: ▶ prompt_template_with_input = """Below is an instruction that describes a task,
        and an input to the instruction. Your goal is to complete the instruction.

        ### Instruction:
        {instruction}

        ### Input:
        {input}

        ### Response: """

        prompt_template_without_input = """Below is an instruction that describes a task.
        Your goal is to complete the instruction.

        ### Instruction:
        {instruction}

        ### Response: """
```

Hydrate prompts (add data to prompts)

```
In [5]: ▶ processed_data = []
        for j in top_m:
            if not j["input"]:
                processed_prompt = prompt_template_without_input.format(instruction=j["instruction"])
            else:
                processed_prompt = prompt_template_with_input.format(instruction=j["instruction"], input=j["input"])
            processed_data.append({"input": processed_prompt, "output": j["output"]})
```

```
In [6]: ▶ pprint(processed_data[0])
```

```
{'input': 'Below is an instruction that describes a task. Write a response '
        'that appropriately completes the request.\n'
        '\n'
        '### Instruction:\n'
        'Give three tips for staying healthy.\n'
        '\n'
        '### Response:',
 'output': '1.Eat a balanced diet and make sure to include plenty of fruits '
        'and vegetables. \n'
        '2. Exercise regularly to keep your body active and strong. \n'
        '3. Get enough sleep and maintain a consistent sleep schedule.'}
```

Save data to jsonl

```
In [7]: ▶ with jsonlines.open(f'alpaca_processed.jsonl', 'w') as writer:
        writer.write_all(processed_data)
```

Compare non-instruction-tuned vs. instruction-tuned models

```
In [8]: ▶ dataset_path_hf = "lamini/alpaca"
        dataset_hf = load_dataset(dataset_path_hf)
        print(dataset_hf)
```

```
Downloading readme: 0%|          | 0.00/388 [00:00<?, ?B/s]
Downloading data files: 0%|          | 0/1 [00:00<?, ?it/s]
Downloading data: 0%|          | 0.00/12.7M [00:00<?, ?B/s]
Extracting data files: 0%|          | 0/1 [00:00<?, ?it/s]
Generating train split: 0%|          | 0/52002 [00:00<?, ? examples/s]
DatasetDict({
  train: Dataset({
    features: ['input', 'output'],
    num_rows: 52002
  })
})
```

```
In [9]: ▶ non_instruct_model = BasicModelRunner("meta-llama/Llama-2-7b-hf")
non_instruct_output = non_instruct_model("Tell me how to train my dog to sit")
print("Not instruction-tuned output (Llama 2 Base):", non_instruct_output)
```

Not instruction-tuned output (Llama 2 Base): Tell me how to train my dog to stay.

Tell me how to teach my dog to come.

Tell me how to get my dog to heel.

Tell me how to stop my dog from jumping.

Tell me how to make my dog stop barking.

Tell me how I can get my dog to stop chewing.

Tell me how to house train my dog.

Tell me how to potty train my dog.

Tell me how to train my dog to walk on a leash.

Tell me how to crate train my dog.

Tell Me How To Train My Dog To Sit, Stay, Come, Heel, Stop Jumping, Stop Barking, Stop Chewing, House Train, Potty Train, Walk On A Leash, Crate Train, And More!

Tell Me How To Train Your Dog To Sit, Stay, Heel, Come, Stop Jumping, Stop Chewing, House Trained, Potty Trained, Walk On A Leash, And More!

Tell me how to train your dog to sit.

Tell Me How To House Train My Dog.

```
In [10]: ▶ instruct_model = BasicModelRunner("meta-llama/Llama-2-7b-chat-hf")
instruct_output = instruct_model("Tell me how to train my dog to sit")
print("Instruction-tuned output (Llama 2): ", instruct_output)
```

Instruction-tuned output (Llama 2): on command. Training your dog to sit is a basic obedience command that can be achieved with patience, consistency, and positive reinforcement. Here's a step-by-step guide on how to train your dog to sit:

1. Choose a quiet and distraction-free area: Find a quiet room or area where your dog can focus on you without getting distracted.
2. Have treats ready: Choose your dog's favorite treats and have them readily available to reward good behavior.
3. Stand in front of your dog: Stand in front of your dog and hold a treat close to their nose.
4. Move the treat up and back: Slowly move the treat upwards and backwards, towards your dog's tail, while saying "sit" in a calm and clear voice.
5. Dog will follow the treat: As you move the treat, your dog should naturally sit down to follow it. The moment they touch their bottom to the ground, give them the treat and praise them.

Note: This section of the notebook has been updated.

Instruction-tuned output (ChatGPT) responds with:

Training your dog to sit is a basic and essential command that can be taught using positive reinforcement. Here's a simple step-by-step guide:

1. **Prepare Treats:** Gather small, soft treats that your dog enjoys. Make sure they are easy to chew and won't take too long to eat.
2. **Find a Quiet Space:** Choose a quiet area with minimal distractions for the training session. This will help your dog focus better.
3. **Get Your Dog's Attention:** Call your dog's name to get their attention. Make sure they are looking at you.
4. **Use a Treat to Lure:** Hold a treat close to your dog's nose, and slowly move your hand upward and slightly backward over their head. As you do this, your dog's natural response will be to follow the treat with their nose, causing them to sit.
5. **Say the Command:** As your dog starts to sit, say the command "Sit" in a clear and firm voice. Use the word consistently every time you want your dog to sit.
6. **Reward and Praise:** As soon as your dog sits, immediately reward them with the treat and offer verbal praise. This positive reinforcement will help them associate sitting with positive outcomes.
7. **Repeat and Practice:** Repeat the process several times in a row during each training session. Keep the sessions short (around 5-10 minutes) to prevent your dog from losing interest.
8. **Add Duration:** Once your dog consistently sits on command, you can gradually increase the duration by waiting a couple of seconds before giving the treat. This helps reinforce the sit command.
9. **Generalize the Command:** Practice the "sit" command in different locations and with various distractions to help your dog generalize the behavior.
10. **Be Patient and Consistent:** Patience and consistency are key in dog training. Always use positive reinforcement, and avoid punishment. If your dog doesn't succeed initially, go back a step and try again.

Try smaller models

```
In [11]: ▶ tokenizer = AutoTokenizer.from_pretrained("EleutherAI/pythia-70m")
model = AutoModelForCausalLM.from_pretrained("EleutherAI/pythia-70m")
```

```
/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
```

```
tokenizer_config.json:  0%|          | 0.00/396 [00:00<?, ?B/s]
```

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
special_tokens_map.json:  0%|          | 0.00/99.0 [00:00<?, ?B/s]
```

Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.

```
config.json:  0%|          | 0.00/567 [00:00<?, ?B/s]
```

```
/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
```

```
model.safetensors:  0%|          | 0.00/166M [00:00<?, ?B/s]
```

```
In [12]: ▶ def inference(text, model, tokenizer, max_input_tokens=1000, max_output_tokens=1000):
# Tokenize
input_ids = tokenizer.encode(
    text,
    return_tensors="pt",
    truncation=True,
    max_length=max_input_tokens
)

# Generate
device = model.device
generated_tokens_with_prompt = model.generate(
    input_ids=input_ids.to(device),
    max_length=max_output_tokens
)

# Decode
generated_text_with_prompt = tokenizer.batch_decode(generated_tokens_with_prompt)

# Strip the prompt
generated_text_answer = generated_text_with_prompt[0][len(text):]

return generated_text_answer
```

```
In [13]: ► finetuning_dataset_path = "lamini/lamini_docs"
finetuning_dataset = load_dataset(finetuning_dataset_path)
print(finetuning_dataset)
```

```
Downloading readme: 0%|          | 0.00/577 [00:00<?, ?B/s]
```

```
Downloading data files: 0%|          | 0/2 [00:00<?, ?it/s]
```

```
Downloading data: 0%|          | 0.00/615k [00:00<?, ?B/s]
```

```
Downloading data: 0%|          | 0.00/83.7k [00:00<?, ?B/s]
```

```
Extracting data files: 0%|          | 0/2 [00:00<?, ?it/s]
```

```
Generating train split: 0%|          | 0/1260 [00:00<?, ? examples/s]
```

```
Generating test split: 0%|          | 0/140 [00:00<?, ? examples/s]
```

```
DatasetDict({
  train: Dataset({
    features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
    num_rows: 1260
  })
  test: Dataset({
    features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
    num_rows: 140
  })
})
```

```
In [14]: ▶ test_sample = finetuning_dataset["test"][0]
print(test_sample)

print(inference(test_sample["question"], model, tokenizer))
```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

[illegible]

I have a question about the following:

How do I get the correct documentation to work?

A:

I think you need to use the following code:

A:

You can use the following code to get the correct documentation.

A:

You can use the following code to get the correct documentation.

A:

You can use the following

Compare to finetuned small model

```
In [15]: ▶ instruction_model = AutoModelForCausalLM.from_pretrained("lamini/lamini_doc
```

```
/usr/local/lib/python3.9/site-packages/huggingface_hub/file_download.py:943: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
```

```
config.json: 0%|          | 0.00/717 [00:00<?, ?B/s]
```

```
model.safetensors: 0%|          | 0.00/282M [00:00<?, ?B/s]
```

```
generation_config.json: 0%|          | 0.00/111 [00:00<?, ?B/s]
```

```
In [16]: ▶ print(inference(test_sample["question"], instruction_model, tokenizer))
```

The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention\_mask` to obtain reliable results.

Setting `pad\_token\_id` to `eos\_token\_id`:0 for open-end generation.

Yes, Lamini can generate technical documentation or user manuals for software projects. This can be achieved by providing a prompt for a specific technical question or question to the LLM Engine, or by providing a prompt for a specific technical question or question. Additionally, Lamini can be trained on specific technical questions or questions to help users understand the process and provide feedback to the LLM Engine. Additionally, Lamini

```
In [17]: ▶ # Pssst! If you were curious how to upload your own dataset to Huggingface
# Here is how we did it

# !pip install huggingface_hub
# !huggingface-cli login

# import pandas as pd
# import datasets
# from datasets import Dataset

# finetuning_dataset = Dataset.from_pandas(pd.DataFrame(data=finetuning_data))
# finetuning_dataset.push_to_hub(dataset_path_hf)
```